

```
In [1]: import os
import numpy as np
import matplotlib.pyplot as plt

from PIL import Image
from tqdm import tqdm

import torch
import torch.nn as nn

from torchvision import transforms
from torchvision.utils import make_grid

from torch.utils.data import Dataset, DataLoader
```

```
In [10]: device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print(device)
```

cpu

```
In [3]: IMAGE_SIZE = 64

LATENT_DIM = 100

BATCH_SIZE = 16

EPOCHS = 50

LR = 0.0002
```

```
In [4]: LONG_HAIR = {
    "madonna": 1,
    "mindy_kaling": 1,

    "ben_afflek": 0,
    "elton_john": 0,
    "jerry_seinfeld": 0
}
```

```
In [5]: class LongHairDataset(Dataset):

    def __init__(self, root_dir, transform=None):

        self.transform = transform
        self.samples = []

        for celebrity in os.listdir(root_dir):

            celeb_path = os.path.join(
                root_dir,
                celebrity
```

```

    )

    if not os.path.isdir(celeb_path):
        continue

    label = LONG_HAIR[celebrity]

    for img in os.listdir(celeb_path):

        if img.lower().endswith(
            ".jpg", ".jpeg", ".png"
        ):

            self.samples.append(
                (
                    os.path.join(
                        celeb_path,
                        img
                    ),
                    label
                )
            )

def __len__(self):
    return len(self.samples)

def __getitem__(self, idx):

    img_path, label = self.samples[idx]

    image = Image.open(
        img_path
    ).convert("RGB")

    if self.transform:
        image = self.transform(image)

    return image, label

```

```

In [6]: transform = transforms.Compose([

    transforms.Resize((64,64)),

    transforms.ToTensor(),

    transforms.Normalize(
        (0.5,0.5,0.5),
        (0.5,0.5,0.5)
    )
])

dataset = LongHairDataset(
    "/home/saidul/Desktop/4-2/Deep Learning/data/5_celebrity_faces/train",
    transform
)

```

```
loader = DataLoader(  
    dataset,  
    batch_size=BATCH_SIZE,  
    shuffle=True  
)  
  
print("Images:", len(dataset))
```

Images: 93

```
In [7]: class Generator(nn.Module):  
  
    def __init__(self):  
        super().__init__()  
  
        self.label_emb = nn.Embedding(  
            2,  
            50  
        )  
  
        self.main = nn.Sequential(  
  
            nn.ConvTranspose2d(  
                150,  
                512,  
                4,  
                1,  
                0,  
                bias=False  
            ),  
  
            nn.BatchNorm2d(512),  
            nn.ReLU(True),  
  
            nn.ConvTranspose2d(  
                512,  
                256,  
                4,  
                2,  
                1,  
                bias=False  
            ),  
  
            nn.BatchNorm2d(256),  
            nn.ReLU(True),  
  
            nn.ConvTranspose2d(  
                256,  
                128,  
                4,  
                2,  
                1,  
                bias=False  
            ),
```

```

        nn.BatchNorm2d(128),
        nn.ReLU(True),

        nn.ConvTranspose2d(
            128,
            64,
            4,
            2,
            1,
            bias=False
        ),

        nn.BatchNorm2d(64),
        nn.ReLU(True),

        nn.ConvTranspose2d(
            64,
            3,
            4,
            2,
            1,
            bias=False
        ),

        nn.Tanh()
    )

    def forward(self, noise, labels):

        label_embedding = self.label_emb(
            labels
        )

        x = torch.cat(
            [noise, label_embedding],
            dim=1
        )

        x = x.view(
            -1,
            150,
            1,
            1
        )

        return self.main(x)

```

In [8]: **class** Discriminator(nn.Module):

```

    def __init__(self):

        super().__init__()

        self.label_emb = nn.Embedding(
            2,
            64*64

```

```

)

self.main = nn.Sequential(

    nn.Conv2d(
        4,
        64,
        4,
        2,
        1
    ),

    nn.LeakyReLU(0.2),

    nn.Conv2d(
        64,
        128,
        4,
        2,
        1
    ),

    nn.BatchNorm2d(128),
    nn.LeakyReLU(0.2),

    nn.Conv2d(
        128,
        256,
        4,
        2,
        1
    ),

    nn.BatchNorm2d(256),
    nn.LeakyReLU(0.2),

    nn.Conv2d(
        256,
        512,
        4,
        2,
        1
    ),

    nn.BatchNorm2d(512),
    nn.LeakyReLU(0.2),

    nn.Conv2d(
        512,
        1,
        4,
        1,
        0
    ),

    nn.Sigmoid()

```

```

    )

    def forward(self, images, labels):

        label_map = self.label_emb(
            labels
        )

        label_map = label_map.view(
            -1,
            1,
            64,
            64
        )

        x = torch.cat(
            [images, label_map],
            dim=1
        )

        return self.main(x).view(-1)

```

```

In [11]: G = Generator().to(device)
        D = Discriminator().to(device)

        criterion = nn.BCELoss()

        optimizerG = torch.optim.Adam(
            G.parameters(),
            lr=LR,
            betas=(0.5,0.999)
        )

        optimizerD = torch.optim.Adam(
            D.parameters(),
            lr=LR,
            betas=(0.5,0.999)
        )

```

```

In [12]: g_losses = []
        d_losses = []

        for epoch in range(EPOCHS):

            for real_images, labels in tqdm(loader):

                real_images = real_images.to(device)

                labels = labels.to(device)

                batch_size = real_images.size(0)

                real_target = torch.ones(
                    batch_size,
                    device=device

```

```

)

fake_target = torch.zeros(
    batch_size,
    device=device
)

#####
# Train D
#####

optimizerD.zero_grad()

real_output = D(
    real_images,
    labels
)

loss_real = criterion(
    real_output,
    real_target
)

noise = torch.randn(
    batch_size,
    LATENT_DIM,
    device=device
)

fake_images = G(
    noise,
    labels
)

fake_output = D(
    fake_images.detach(),
    labels
)

loss_fake = criterion(
    fake_output,
    fake_target
)

lossD = loss_real + loss_fake

lossD.backward()

optimizerD.step()

#####
# Train G
#####

optimizerG.zero_grad()

```

```

        output = D(
            fake_images,
            labels
        )

        lossG = criterion(
            output,
            real_target
        )

        lossG.backward()

        optimizerG.step()

    g_losses.append(lossG.item())
    d_losses.append(lossD.item())

    print(
        f"Epoch [{epoch+1}/{EPOCHS}] "
        f"D={lossD.item():.4f} "
        f"G={lossG.item():.4f}"
    )

```

```

100%|██████████| 6/6 [00:03<00:00, 1.98it/s]
Epoch [1/50] D=1.5221 G=3.9910

```

```

100%|██████████| 6/6 [00:03<00:00, 1.72it/s]
Epoch [2/50] D=0.8676 G=7.1206

```

```

100%|██████████| 6/6 [00:03<00:00, 1.88it/s]
Epoch [3/50] D=1.0063 G=4.3516

```

```

100%|██████████| 6/6 [00:02<00:00, 2.03it/s]
Epoch [4/50] D=1.5254 G=8.0166

```

```

100%|██████████| 6/6 [00:03<00:00, 1.97it/s]
Epoch [5/50] D=1.3674 G=3.5421

```

```

100%|██████████| 6/6 [00:04<00:00, 1.50it/s]
Epoch [6/50] D=1.4749 G=4.9953

```

```

100%|██████████| 6/6 [00:03<00:00, 1.76it/s]
Epoch [7/50] D=0.7470 G=3.8735

```

```

100%|██████████| 6/6 [00:03<00:00, 1.75it/s]
Epoch [8/50] D=1.0296 G=6.5747

```

```

100%|██████████| 6/6 [00:03<00:00, 1.62it/s]
Epoch [9/50] D=0.4206 G=3.7613

```

```

100%|██████████| 6/6 [00:03<00:00, 1.55it/s]
Epoch [10/50] D=1.2591 G=7.0753

```

```

100%|██████████| 6/6 [00:03<00:00, 1.82it/s]
Epoch [11/50] D=0.8790 G=5.5056

```

```

100%|██████████| 6/6 [00:04<00:00, 1.41it/s]
Epoch [12/50] D=0.4778 G=5.6716

```

```

100%|██████████| 6/6 [00:03<00:00, 1.61it/s]
Epoch [13/50] D=0.3172 G=4.7066

```

```

100%|██████████| 6/6 [00:03<00:00, 1.68it/s]
Epoch [14/50] D=1.5424 G=4.8875

```

```

100%|██████████| 6/6 [00:03<00:00, 1.78it/s]

```


Epoch [15/50] D=0.7581 G=4.5275
100%|██████████| 6/6 [00:03<00:00, 1.62it/s]
Epoch [16/50] D=0.5934 G=2.7372
100%|██████████| 6/6 [00:03<00:00, 1.54it/s]
Epoch [17/50] D=2.0317 G=3.4504
100%|██████████| 6/6 [00:03<00:00, 1.55it/s]
Epoch [18/50] D=0.4508 G=2.7633
100%|██████████| 6/6 [00:03<00:00, 1.69it/s]
Epoch [19/50] D=1.3094 G=5.9881
100%|██████████| 6/6 [00:03<00:00, 1.69it/s]
Epoch [20/50] D=1.3235 G=7.9869
100%|██████████| 6/6 [00:03<00:00, 1.53it/s]
Epoch [21/50] D=1.1813 G=2.6401
100%|██████████| 6/6 [00:03<00:00, 1.67it/s]
Epoch [22/50] D=0.9441 G=6.4744
100%|██████████| 6/6 [00:03<00:00, 1.65it/s]
Epoch [23/50] D=1.6738 G=7.3825
100%|██████████| 6/6 [00:03<00:00, 1.60it/s]
Epoch [24/50] D=0.6548 G=3.9847
100%|██████████| 6/6 [00:03<00:00, 1.60it/s]
Epoch [25/50] D=1.2229 G=5.5519
100%|██████████| 6/6 [00:04<00:00, 1.46it/s]
Epoch [26/50] D=1.1142 G=4.7680
100%|██████████| 6/6 [00:03<00:00, 1.76it/s]
Epoch [27/50] D=0.6773 G=2.9380
100%|██████████| 6/6 [00:02<00:00, 2.23it/s]
Epoch [28/50] D=0.4034 G=2.9948
100%|██████████| 6/6 [00:02<00:00, 2.13it/s]
Epoch [29/50] D=1.0251 G=3.5086
100%|██████████| 6/6 [00:03<00:00, 1.79it/s]
Epoch [30/50] D=0.4295 G=3.7207
100%|██████████| 6/6 [00:04<00:00, 1.48it/s]
Epoch [31/50] D=0.5212 G=6.0403
100%|██████████| 6/6 [00:03<00:00, 1.62it/s]
Epoch [32/50] D=0.7850 G=4.7745
100%|██████████| 6/6 [00:03<00:00, 1.71it/s]
Epoch [33/50] D=0.7629 G=8.8128
100%|██████████| 6/6 [00:03<00:00, 1.59it/s]
Epoch [34/50] D=0.6992 G=6.0581
100%|██████████| 6/6 [00:04<00:00, 1.43it/s]
Epoch [35/50] D=1.4973 G=7.3370
100%|██████████| 6/6 [00:03<00:00, 1.66it/s]
Epoch [36/50] D=0.8972 G=3.1601
100%|██████████| 6/6 [00:03<00:00, 1.71it/s]
Epoch [37/50] D=0.8312 G=3.2322
100%|██████████| 6/6 [00:03<00:00, 1.80it/s]
Epoch [38/50] D=0.3855 G=5.2062
100%|██████████| 6/6 [00:03<00:00, 1.88it/s]
Epoch [39/50] D=0.4146 G=3.6858

```
100%|██████████| 6/6 [00:03<00:00, 1.79it/s]
Epoch [40/50] D=0.3196 G=3.7519
100%|██████████| 6/6 [00:03<00:00, 1.89it/s]
Epoch [41/50] D=0.2152 G=4.7620
100%|██████████| 6/6 [00:03<00:00, 1.84it/s]
Epoch [42/50] D=0.4868 G=4.9729
100%|██████████| 6/6 [00:03<00:00, 1.88it/s]
Epoch [43/50] D=0.9923 G=6.6821
100%|██████████| 6/6 [00:03<00:00, 1.64it/s]
Epoch [44/50] D=0.8782 G=4.0450
100%|██████████| 6/6 [00:03<00:00, 1.93it/s]
Epoch [45/50] D=0.3983 G=4.4249
100%|██████████| 6/6 [00:03<00:00, 1.95it/s]
Epoch [46/50] D=0.6287 G=3.4830
100%|██████████| 6/6 [00:03<00:00, 1.78it/s]
Epoch [47/50] D=0.5153 G=4.0313
100%|██████████| 6/6 [00:03<00:00, 1.92it/s]
Epoch [48/50] D=1.1003 G=7.6555
100%|██████████| 6/6 [00:03<00:00, 1.73it/s]
Epoch [49/50] D=0.9936 G=4.5438
100%|██████████| 6/6 [00:03<00:00, 1.90it/s]
Epoch [50/50] D=0.3923 G=3.9542
```

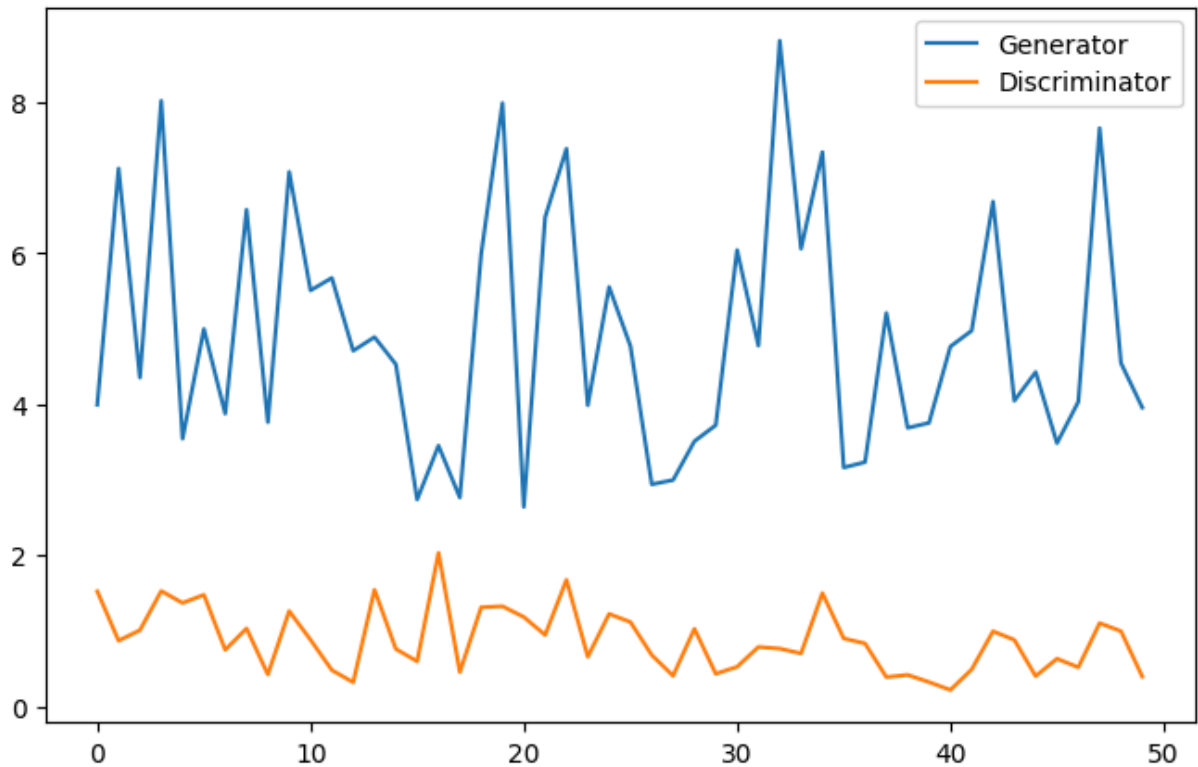
```
In [13]: plt.figure(figsize=(8,5))

plt.plot(
    g_losses,
    label="Generator"
)

plt.plot(
    d_losses,
    label="Discriminator"
)

plt.legend()

plt.show()
```



```
In [14]: G.eval()

with torch.no_grad():

    labels = torch.ones(
        16,
        dtype=torch.long,
        device=device
    )

    noise = torch.randn(
        16,
        LATENT_DIM,
        device=device
    )

    fake_long = G(
        noise,
        labels
    ).cpu()
```

```
In [15]: grid = make_grid(
    fake_long,
    nrow=4,
    normalize=True
)

plt.figure(figsize=(8,8))

plt.imshow(
    np.transpose(
```

```

        grid.numpy(),
        (1,2,0)
    )
)

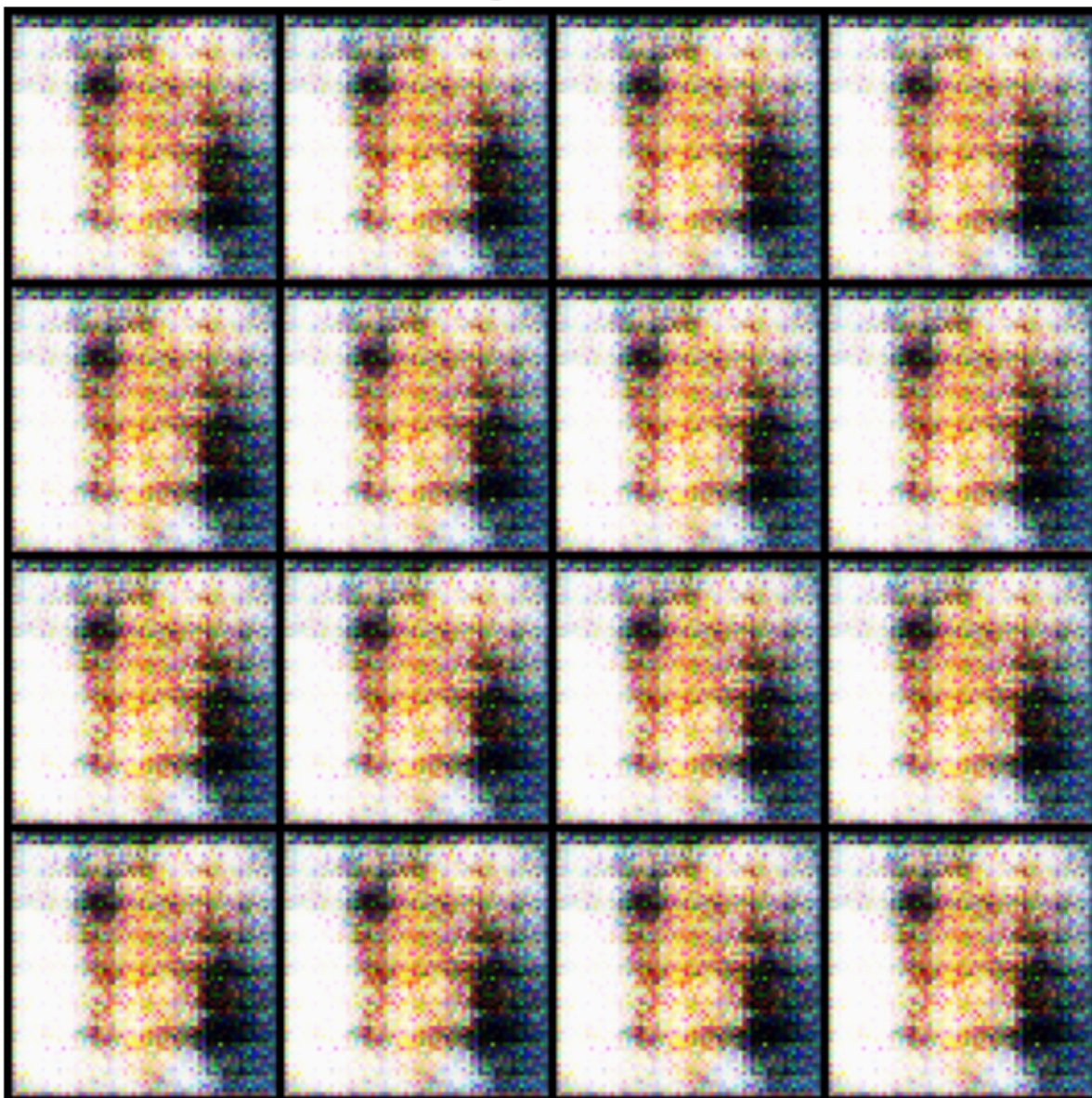
plt.title("Long Hair Faces")

plt.axis("off")

plt.show()

```

Long Hair Faces



```

In [16]: with torch.no_grad():

        labels = torch.zeros(
            16,
            dtype=torch.long,
            device=device
        )

```

```
noise = torch.randn(
    16,
    LATENT_DIM,
    device=device
)

fake_short = G(
    noise,
    labels
).cpu()
```

```
In [17]: grid = make_grid(
    fake_short,
    nrow=4,
    normalize=True
)

plt.figure(figsize=(8,8))

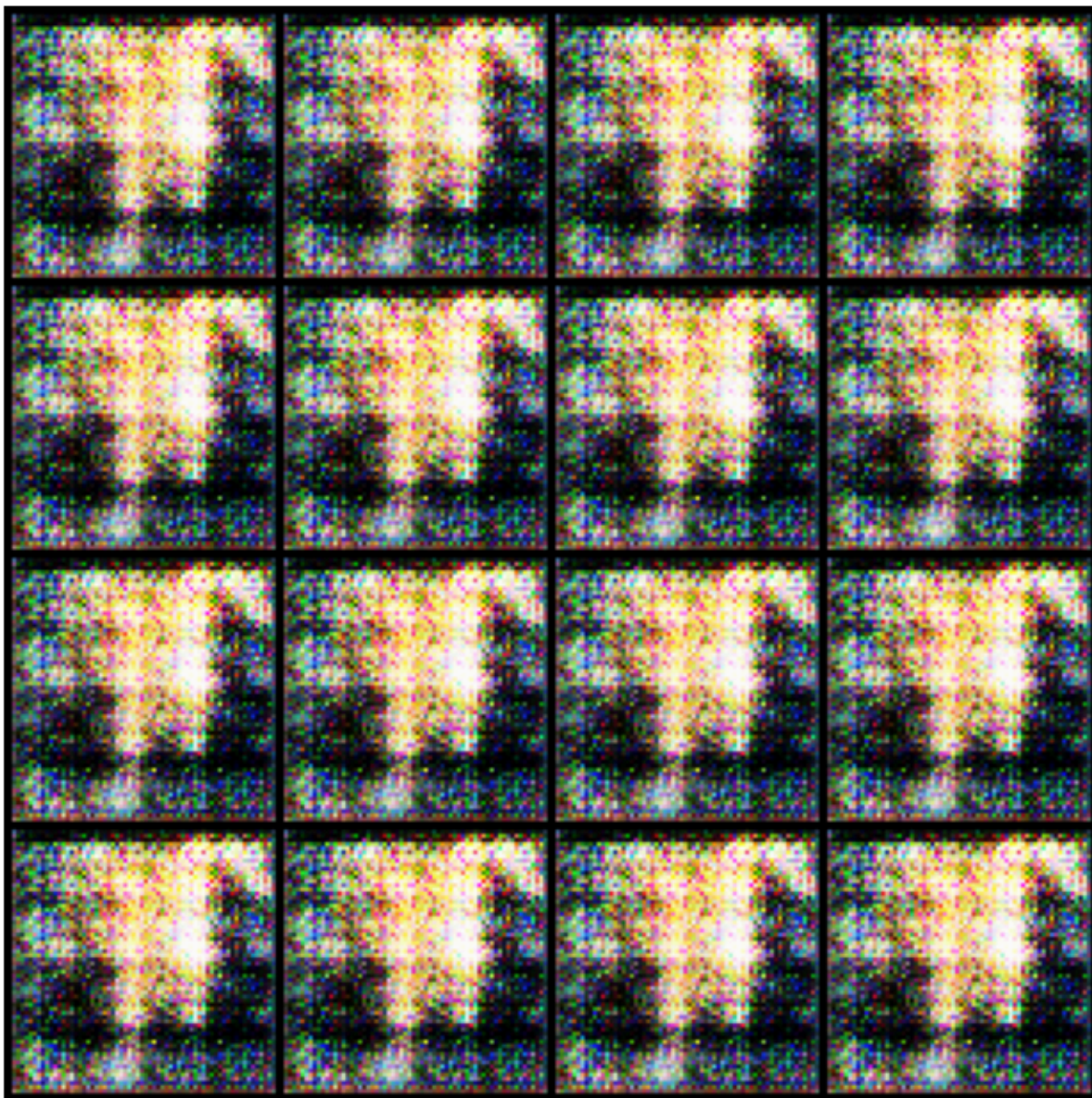
plt.imshow(
    np.transpose(
        grid.numpy(),
        (1,2,0)
    )
)

plt.title("Short Hair Faces")

plt.axis("off")

plt.show()
```


Short Hair Faces



```
In [18]: from torchvision.utils import make_grid

long_imgs = []
short_imgs = []

for img, label in dataset:

    if label == 1 and len(long_imgs) < 8:
        long_imgs.append(img)

    if label == 0 and len(short_imgs) < 8:
        short_imgs.append(img)

plt.figure(figsize=(8,4))
plt.imshow(
    np.transpose(
        make_grid(long_imgs, nrow=4, normalize=True).numpy(),
        (1,2,0)
    )
)
```

```

    )
)
plt.title("Real Long Hair Examples")
plt.axis("off")
plt.show()

plt.figure(figsize=(8,4))
plt.imshow(
    np.transpose(
        make_grid(short_imgs, nrow=4, normalize=True).numpy(),
        (1,2,0)
    )
)
plt.title("Real Short Hair Examples")
plt.axis("off")
plt.show()

```

Real Long Hair Examples



Real Short Hair Examples

